# Project Planning Phase

## Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

| | |
|---|---|
| **Date** | 04 October 2026 |
| **Team ID** | SWTID-2026-5690 |
| **Project Name** | Implement Client Script & UI Policy (Incident) |
| **Team Lead** | Rekha J |
| **Team Member** | Afrit Khan R |
| **Maximum Marks** | 5 Marks |

## Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| Sprint-1 | Project Setup & Requirement Analysis | USN-1 | As a developer, I can set up a ServiceNow Personal Developer Instance (PDI) and log in as an administrator to build and test Incident form customizations. | 2 | High | Rekha J |
| Sprint-1 | | USN-2 | As an administrator, I can create a custom Update Set so that all Client Script and UI Policy changes are tracked and portable. | 1 | High | Afrit Khan R |
| Sprint-1 | | USN-3 | As an administrator, I can create test users and assign roles (itil, admin) so that form behavior can be validated for different users. | 2 | Medium | Afrit Khan R |
| Sprint-1 | | USN-4 | As a team, we can analyse the Incident table fields (Caller, Category, Subcategory, Impact, Urgency, Priority, State, Assignment group) and document the client-side behavior requirements. | 3 | High | Rekha J, Afrit Khan R |
| Sprint-1 | UI Policy (Incident) | USN-5 | As a service desk agent, I am required to fill in Short description and Category when creating an incident, so that every incident record is complete. | 3 | High | Rekha J |
| Sprint-1 | | USN-6 | As an agent, I see the Subcategory field only after a Category is selected, so that the form stays clean and easy to use. | 3 | High | Afrit Khan R |
| Sprint-1 | | USN-7 | As an agent, I must enter Close notes when the State is set to Resolved, so that every resolution is documented. | 3 | Medium | Rekha J |
| Sprint-1 | | USN-8 | As an agent, I cannot edit the Caller field once the incident has been saved, so that the original requester is preserved. | 3 | Medium | Afrit Khan R |
| Sprint-2 | Client Script (Incident) | USN-9 | As an agent, I see an information message when a new incident form loads (onLoad), reminding me to enter complete details. | 2 | Medium | Rekha J |
| Sprint-2 | | USN-10 | As an agent, I see Contact type and Urgency pre-populated with default values when I open a new incident (onLoad). | 2 | Medium | Afrit Khan R |
| Sprint-2 | | USN-11 | As an agent, when I change the Category (onChange), the Subcategory value is cleared so that no mismatched combination is saved. | 3 | High | Rekha J |
| Sprint-2 | | USN-12 | As an agent, I get an alert when both Impact and Urgency are set to High (onChange), so that critical incidents are escalated quickly. | 3 | High | Afrit Khan R |

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| Sprint-2 | | USN-13 | As an agent, I am prevented from saving an incident when the Short description is shorter than 10 characters (onSubmit). | 3 | High | Rekha J |
| Sprint-2 | | USN-14 | As an agent, I am asked to confirm before resolving or closing an incident (onSubmit), so that accidental closures are avoided. | 3 | Medium | Afrit Khan R |
| Sprint-2 | | USN-15 | As an agent, I see a notification when I select a VIP Caller (onChange using g_form.getReference), so that I can prioritise accordingly. | 4 | Medium | Rekha J, Afrit Khan R |
| Sprint-3 | UI Policy (Advanced) | USN-16 | As an agent, I see the On hold reason field, and it becomes mandatory, when the State is set to On Hold. | 3 | High | Rekha J |
| Sprint-3 | | USN-17 | As an agent, the Assignment group becomes mandatory when Priority is 1 - Critical, so that urgent incidents are never left unassigned. | 3 | High | Afrit Khan R |
| Sprint-3 | | USN-18 | As an agent, the resolution-related fields become read-only when the State is Closed, using UI Policy conditions and scripts. | 3 | Medium | Rekha J |
| Sprint-3 | | USN-19 | As an administrator, I configure On load and Reverse if false options so that UI Policy actions are undone when the condition no longer applies. | 2 | Medium | Afrit Khan R |
| Sprint-3 | Client Script & UI Policy Integration | USN-20 | As a developer, I can set UI Policy order and review Client Script execution so that the two do not conflict on the Incident form. | 3 | High | Rekha J, Afrit Khan R |
| Sprint-3 | | USN-21 | As an agent, I can edit State and Impact from the Incident list view with an onCellEdit Client Script validating the change. | 3 | Low | Afrit Khan R |
| Sprint-3 | | USN-22 | As a user, I get consistent UI Policy behavior across the standard form view and other UI types where the policy is enabled. | 3 | Low | Rekha J |
| Sprint-4 | Testing & Validation | USN-23 | As a tester, I can run positive and negative test cases for every UI Policy using different user roles. | 4 | High | Rekha J |
| Sprint-4 | | USN-24 | As a tester, I can verify each Client Script (onLoad, onChange, onSubmit) in the browser and confirm there are no console errors. | 4 | High | Afrit Khan R |
| Sprint-4 | | USN-25 | As a tester, I can run an end-to-end regression of the incident lifecycle (create, assign, hold, resolve, close) to confirm standard behavior is not broken. | 3 | High | Rekha J, Afrit Khan R |
| Sprint-4 | Documentation & Deployment | USN-26 | As a team, we can document the scripts, UI Policy conditions and test results for the Incident form customizations. | 3 | Medium | Rekha J |
| Sprint-4 | | USN-27 | As an administrator, I can preview and commit the Update Set to a target instance to deploy the customizations. | 3 | Medium | Afrit Khan R |
| Sprint-4 | | USN-28 | As a team, we can prepare the final demonstration and project report. | 3 | Medium | Rekha J, Afrit Khan R |

**Total story points: 80 (4 sprints x 20 story points)**

## Project Tracker, Velocity & Burndown Chart: (4 Marks)

| Sprint | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed (as on Planned End Date) | Sprint Release Date (Actual) |
|---|---|---|---|---|---|---|
| Sprint-1 | 20 | 6 Days | 05 Oct 2026 | 10 Oct 2026 | | |
| Sprint-2 | 20 | 6 Days | 12 Oct 2026 | 17 Oct 2026 | | |
| Sprint-3 | 20 | 6 Days | 19 Oct 2026 | 24 Oct 2026 | | |
| Sprint-4 | 20 | 6 Days | 26 Oct 2026 | 31 Oct 2026 | | |
| **Total** | **80** | **24 Days** | | | | |

### Velocity:

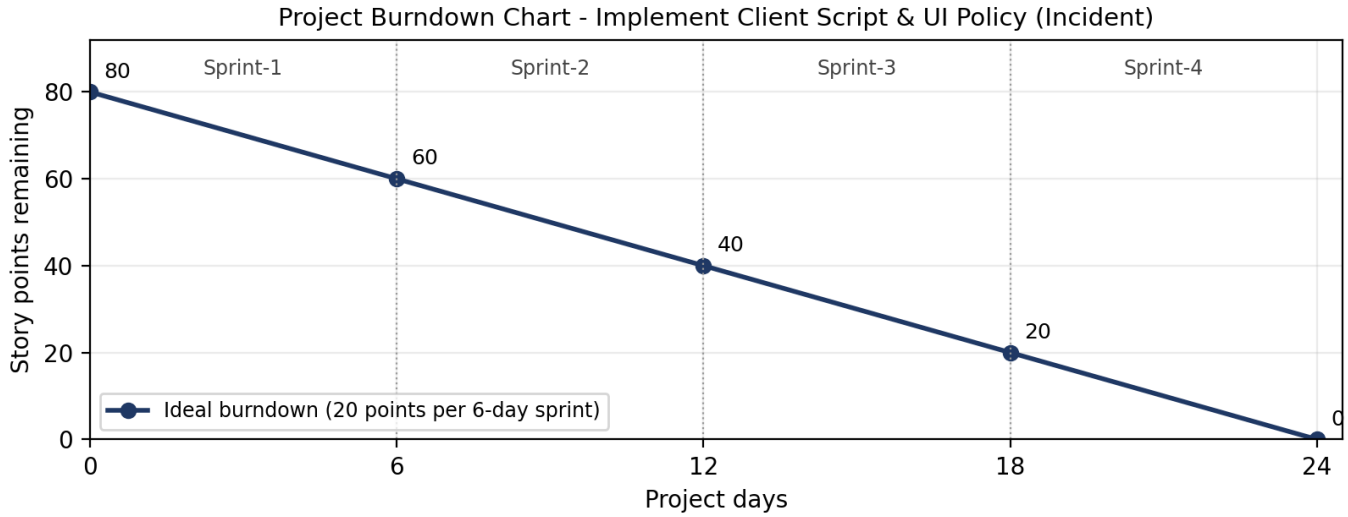
Our sprint duration is 6 days and the velocity of the team is 20 (story points per sprint). Let's calculate the team's average velocity (AV) per iteration unit (story points per day).

$$AV = \frac{velocity}{sprint\ duration} = \frac{20}{6} \approx 3.33$$

The team therefore plans to complete an average of about 3.33 story points per day. Across the project: 80 story points over 24 days is also about 3.33 story points per day.

## Burndown Chart:

A burn down chart is a graphical representation of work left to do versus time. It is often used in agile software development methodologies such as Scrum. However, burn down charts can be applied to any project containing measurable progress over time.




Remaining story points at each sprint end (planned): Sprint-1: 60, Sprint-2: 40, Sprint-3: 20, Sprint-4: 0. The actual burndown is to be plotted against this ideal line as sprints are completed.

https://www.visual-paradigm.com/scrum/scrum-burndown-chart/
https://www.atlassian.com/agile/tutorials/burndown-charts

## Reference:


https://www.atlassian.com/agile/project-management
https://www.atlassian.com/agile/tutorials/how-to-do-scrum-with-jira-software
https://www.atlassian.com/agile/tutorials/epics
https://www.atlassian.com/agile/tutorials/sprints
https://www.atlassian.com/agile/project-management/estimation
https://www.atlassian.com/agile/tutorials/burndown-charts